Student Information System

Software Modeling & Design — Weekly Assignment

Week Tasks:

1 · Design Patterns Analysis
2 · Full Class Diagram

GitHub Repository: github.com/eni6667/Student-Information-System

Course: Software Modeling and Design

Part 1 — Design Patterns

Design patterns are reusable solutions to commonly occurring problems in software design. They represent best practices evolved over time by experienced developers. The following three design patterns have been identified as naturally present in the Student Information System (SIS) and are justified based on the system's structure, actors, and feature requirements.

creation of user objects. The system has three distinct actor types — Student, Teacher, and Admin — all of which extend the abstract
g) method that instantiates the correct concrete subclass based on the role string provided.

tion logic would be scattered throughout the system using verbose if/else or switch blocks every time a user needs to be instantiated.
ncapsulates this logic in a single place, making the code more maintainable, extensible (adding a new role like 'Librarian' only requires o
inciple: the system is open for extension but closed for modification.

```
ng type) {

dent();
cher();
();
mentException();
```

Report class, which is responsible for generating and exporting academic reports. It is also applicable to a central SystemManager or St
ades at runtime.

nstances simultaneously could lead to inconsistent data snapshots — one instance might have a different list of students than another
he application lifecycle, accessed via a static getInstance() method. This also reduces memory overhead since the object is created
urrent instances would risk duplicate student records, inconsistent grade assignments, and concurrency conflicts. The Singleton p
hat is easy to test and mock.

```
structor
 {
 students) { ... }
```

ade assignment workflow. When a Teacher calls assignGrade() on a Grade object, the Student who owns that grade needs to be notifie
rsight or reporting. The Grade class acts as the Observable (subject), while Student and Admin act as Observers.

cher.assignGrade() method would need to directly call notification logic on Student and Admin objects, creating tight coupling betwe
bserver pattern decouples the grade-assigning logic from the notification logic: the Grade class simply fires a notification event, and an
uires zero changes to the Grade class. This is directly reflected in the system's swimlane diagram for 'Assign Grade', where the notificat

```
ers = new ArrayList<>();
ver o) { observers.add(o); }

Updated(this));

{ ... }
```

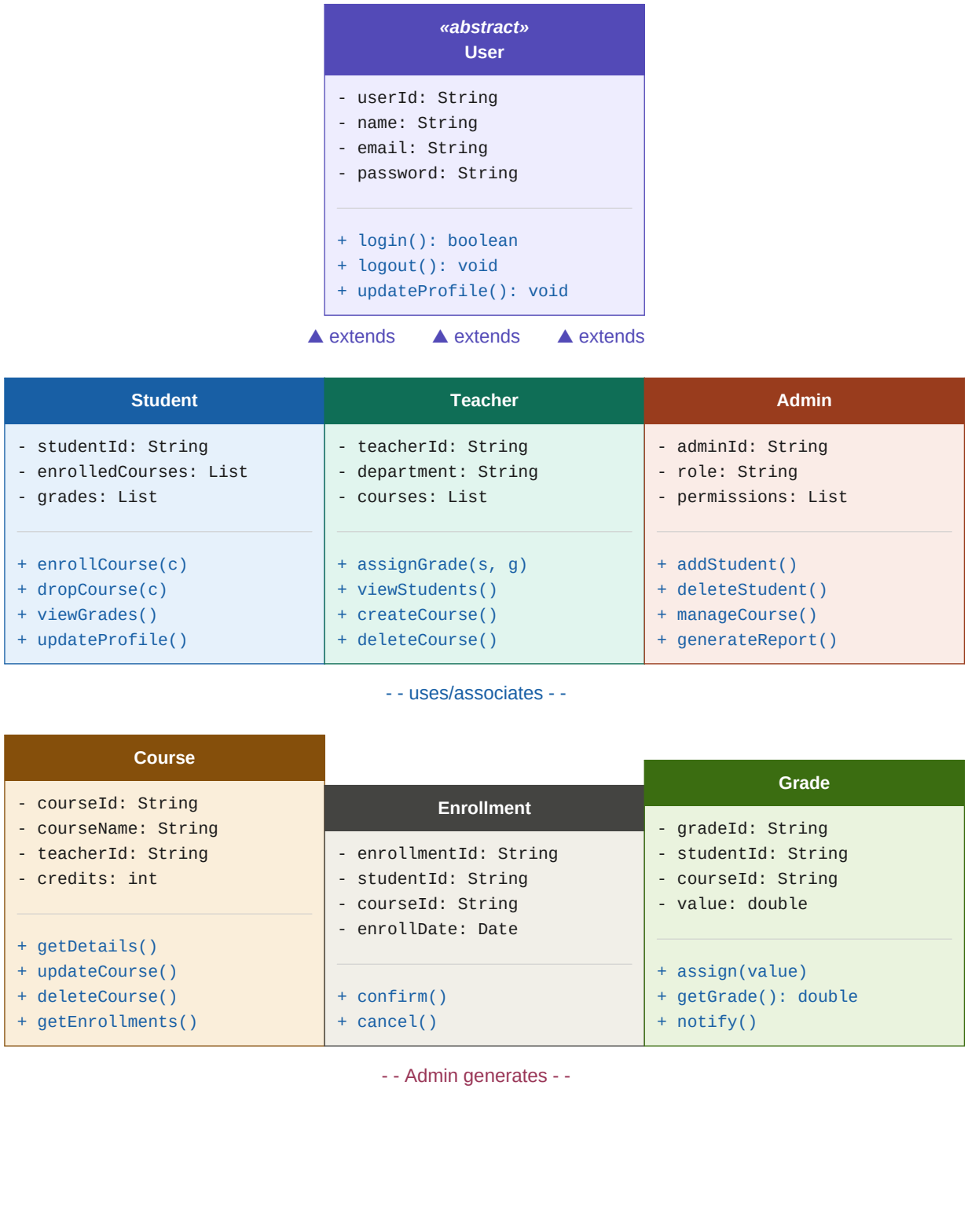
Pattern Summary

Pattern	Category	Applied To	Key Benefit
Factory Method	Creational	UserFactory	Decouples creation from usage
Singleton	Creational	Report, StudentManager	Single shared instance

Observer	Behavioral	Grade → Student/Admin	Decoupled notifications

Part 2 — Class Diagram

The class diagram below presents the complete object-oriented structure of the Student Information System. It shows all identified classes, their attributes, methods, inheritance relationships, and key associations. The diagram follows standard UML notation and incorporates the three design patterns identified in Part 1.



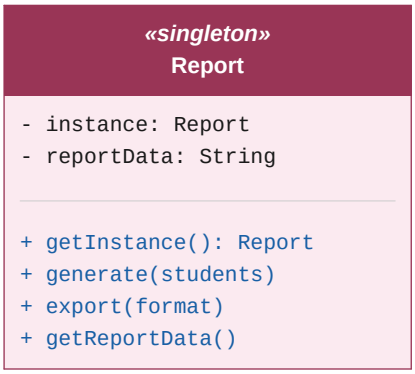


Figure 1 — UML Class Diagram of the Student Information System

Relationship Notes

Relationship	Type	Explanation
User → Student / Teacher / Admin	Inheritance	All user types extend the abstract User class, inheriting login(), logout(), and updateProfile().
Student → Enrollment	Association	A Student can have multiple Enrollments. Each Enrollment links a Student to a Course.
Teacher → Course	Association	A Teacher manages one or more Courses. Teacher.createCourse() instantiates Course objects.
Teacher → Grade	Association	A Teacher assigns Grade objects to Students. Grade.assign() triggers the Observer notification.
Enrollment → Course	Association	Enrollment references a Course by courseId, forming the many-to-many bridge.
Admin → Report	Dependency	Admin calls Report.getInstance() (Singleton) to generate system-wide reports.

Class Descriptions

User (abstract) — The abstract base class for all system actors. Defines shared attributes (userId, name, email, password) and core behaviour (login, logout, updateProfile). Cannot be instantiated directly — the Factory Method pattern ensures only concrete subclasses are created.

Student — Represents a student user. Holds a list of enrolled courses and received grades. Key operations include enrollCourse(), dropCourse(), and viewGrades(). Implements GradeObserver to receive notifications when a grade is assigned.

Teacher — Represents a teaching staff member. Can create and manage courses, view enrolled students, and assign grades. The assignGrade() method triggers the Observer notification chain.

Admin — System administrator with elevated permissions. Responsible for managing the student registry, overseeing course lifecycle, and generating reports via the Singleton Report instance.

Course	Represents an academic course with an ID, name, and associated teacher. Tracks enrollments and provides CRUD operations. A Course is created by a Teacher and managed by Admin.
Enrollment	An association class linking a Student and a Course. Records the enrollment date and status. confirm() and cancel() handle the enrollment lifecycle transitions.
Grade	Represents a grade assigned to a Student for a specific Course. Implements the Observable side of the Observer pattern — the assign() method updates the value and notifies all registered observers (Student, Admin).
Report (Singleton)	A Singleton class responsible for generating and exporting academic reports. getInstance() ensures only one instance exists at runtime. generate() aggregates student and grade data; export() outputs in the requested format (PDF, CSV).